# LQR Control of a Vibrating Square Membrane

Project Handout

Marek Rychlik

March 15, 2026

## Project summary

We consider a square membrane with small transverse displacement. The membrane is fixed on its boundary and is actuated by pressing on a single localized spot. The goal is to design a linear quadratic regulator (LQR) that drives the membrane to rest.

This project begins with a partial differential equation and reduces it to a finite-dimensional control system by expanding in Laplacian eigenfunctions. The finite-dimensional truncation is then treated with standard LQR machinery.

## 1 PDE model

Let

$$\Omega = (0,1) \times (0,1)$$

be the unit square. Let $u(x,y,t)$ denote the vertical displacement of the membrane. For small displacements, $u$ satisfies the controlled wave equation

$$u_{tt}(x,y,t) = c^2 \Delta u(x,y,t) + b(t)\,\psi(x,y), \qquad (x,y) \in \Omega,\ t > 0, \tag{1}$$

with fixed boundary conditions

$$u(x,y,t) = 0, \qquad (x,y) \in \partial\Omega, \tag{2}$$

and initial data

$$u(x,y,0) = u_0(x,y), \qquad u_t(x,y,0) = v_0(x,y). \tag{3}$$

Here $c > 0$ is the wave speed, $b(t) \in \mathbb{R}$ is the scalar control input, and $\psi(x,y)$ is the actuator profile.

The ideal model of pressing at one point $(x_0, y_0) \in \Omega$ is obtained by taking

$$\psi(x,y) = \delta(x - x_0)\delta(y - y_0),$$

but this is singular. For numerical work, it is better to use a localized patch actuator, for example, a narrow Gaussian or a bump function centered at $(x_0, y_0)$.

**Remark.** *For this project, the point-actuator model is useful for the mathematics, while a sharply localized smooth profile is better for computation.*

## 2 Eigenfunction expansion on the square

The Dirichlet Laplacian eigenfunctions on the unit square are

$$\phi_{mn}(x, y) = 2\sin(m\pi x)\sin(n\pi y), \qquad m, n \geq 1, \tag{4}$$

with eigenvalues

$$\lambda_{mn} = \pi^2(m^2 + n^2). \tag{5}$$

These eigenfunctions form an orthonormal basis of $L^2(\Omega)$.

We expand

$$u(x, y, t) = \sum_{m=1}^{\infty}\sum_{n=1}^{\infty} q_{mn}(t)\,\phi_{mn}(x, y). \tag{6}$$

Substituting (6) into (1) and projecting onto mode $(m, n)$ gives

$$\ddot{q}_{mn}(t) + c^2\lambda_{mn}q_{mn}(t) = \beta_{mn}\,b(t), \tag{7}$$

where

$$\beta_{mn} = \int_0^1\int_0^1 \psi(x, y)\phi_{mn}(x, y)\;dx\;dy. \tag{8}$$

If we use the ideal point actuator at $(x_0, y_0)$, then

$$\beta_{mn} = \phi_{mn}(x_0, y_0) = 2\sin(m\pi x_0)\sin(n\pi y_0). \tag{9}$$

Thus every retained mode behaves like a harmonic oscillator driven by the same scalar input $b(t)$, but with mode-dependent gain $\beta_{mn}$.

## 3 Finite-dimensional truncation

Choose an integer $M \geq 1$ and retain modes with

$$1 \leq m, n \leq M.$$

There are then $N = M^2$ retained modes. Flatten the double index $(m, n)$ into a single index $k = 1, \ldots, N$, and write

$$q(t) = \begin{bmatrix} q_1(t) \\ \vdots \\ q_N(t) \end{bmatrix}, \qquad p(t) = \dot{q}(t).$$

Define

$$\omega_k^2 = c^2\lambda_k, \qquad \beta = \begin{bmatrix} \beta_1 \\ \vdots \\ \beta_N \end{bmatrix}, \qquad \Omega_N = \mathrm{diag}(\omega_1^2, \ldots, \omega_N^2).$$

Then the truncated system becomes

$$\dot{x} = Ax + Bb, \qquad x = \begin{bmatrix} q \\ p \end{bmatrix} \in \mathbb{R}^{2N}, \tag{10}$$

with

$$A = \begin{bmatrix} 0 & I \\ -\Omega_N & 0 \end{bmatrix}, \qquad B = \begin{bmatrix} 0 \\ \beta \end{bmatrix}. \tag{11}$$

# 4 Energy and LQR cost

The truncated mechanical energy is

$$E(t) = \frac{1}{2}\sum_{k=1}^{N}\left(p_k(t)^2 + \omega_k^2 q_k(t)^2\right). \tag{12}$$

A natural infinite-horizon quadratic cost is

$$J(b) = \int_0^\infty \left(x(t)^T Q x(t) + R\, b(t)^2\right)\, dt, \tag{13}$$

where $Q \succeq 0$ and $R > 0$.

A physically meaningful choice is

$$Q = \begin{bmatrix} \alpha\,\Omega_N & 0 \\ 0 & \beta_v I \end{bmatrix}, \qquad \alpha > 0,\ \beta_v > 0, \tag{14}$$

so that displacement is penalized according to modal potential energy and velocity is penalized according to kinetic energy.

# 5 LQR feedback law

For the finite-dimensional system (10), the infinite-horizon LQR feedback has the form

$$b(t) = -Kx(t), \tag{15}$$

where

$$K = R^{-1}B^T P, \tag{16}$$

and $P$ is the symmetric positive semidefinite solution of the algebraic Riccati equation

$$A^T P + PA - PBR^{-1}B^T P + Q = 0. \tag{17}$$

The closed-loop dynamics are

$$\dot{x} = (A - BK)x. \tag{18}$$

If the truncated pair $(A, B)$ is stabilizable and $(Q^{1/2}, A)$ is detectable, then the closed-loop matrix $A - BK$ is stable and the modal amplitudes decay to zero.

## 5.1 Note on Riccati equation and "direct method"

Riccati equation will not be used in this project. You will use the "direct method" which only relies upon the ability to find eigenvalues. However, you need to be familiar with Riccati equation, as it is often the best way to solve small examples, such as ones that often appear on exams.

## 6 Why actuator location matters

The actuator enters mode $(m, n)$ through the factor

$$\beta_{mn} = 2\sin(m\pi x_0)\sin(n\pi y_0)$$

for the point-actuator model.

If $\beta_{mn} = 0$, then that mode is not directly actuated. This is especially important for symmetric actuator locations. At the center of the square,

$$(x_0, y_0) = \left(\frac{1}{2}, \frac{1}{2}\right),$$

we get

$$\beta_{mn} = 2\sin\left(\frac{m\pi}{2}\right)\sin\left(\frac{n\pi}{2}\right),$$

which vanishes whenever $m$ or $n$ is even. Thus a centered actuator misses many modes.

For this reason, a good default choice is an off-center actuator such as

$$(x_0, y_0) = (0.37, 0.61).$$

That breaks symmetry and typically produces nonzero coupling to the first many modes.

## 7 A concrete model for the students

A convenient starter model is:

- membrane domain $\Omega = (0, 1)^2$,
- wave speed $c = 1$,
- actuator location $(x_0, y_0) = (0.37, 0.61)$,
- truncation size $M = 6$ or $M = 8$,
- point-actuator couplings from (9),
- LQR weights from (14) with, say, $\alpha = 1$, $\beta_v = 1$, $R = 10^{-2}$ or $10^{-1}$.

## 8 Repository guide

The starter repository includes a Python path. In the Python code, the main driver scripts are:

- `src/python/run_demo.py` for a first closed-loop simulation,
- `src/python/scan_actuator.py` for comparing actuator locations,
- `src/python/modal_lqr.py` for model construction, LQR design, and membrane reconstruction.

The repository also includes a task list with milestones. Students are encouraged to keep their code modular so that changing the truncation size, actuator location, or cost weights requires only a few edits.

## 9 Project tasks

Possible project goals include:

1. Derive the modal equations from the PDE.
2. Build the truncated matrices $A$ and $B$.
3. Compute the LQR gain $K$.
4. Simulate open-loop and closed-loop trajectories for selected initial conditions.
5. Reconstruct the membrane shape from the modal coefficients.
6. Investigate the role of actuator location.
7. Compare a point actuator with a smooth localized actuator patch.

## 10 Possible research questions

A mathematically meaningful research question is:

> How does the location of a single localized actuator affect controllability and LQR stabilization of a vibrating square membrane?

More specific directions:

- What modes are lost when the actuator is placed at the center?
- How does the choice of $R$ affect the tradeoff between fast stabilization and control effort?
- How large should the truncation be before the observed behavior becomes qualitatively stable?
- Does a regularized actuator patch improve numerical robustness compared with the ideal point-actuator model?

## 11 Suggested deliverables

A final submission should include at least the following core items:

- a derivation of the finite-dimensional model,
- code that constructs the LQR controller,
- plots of modal amplitudes and control input,
- snapshots or an animation of the membrane displacement,
- a short discussion of actuator placement and uncontrollable modes.

### 11.1 IMPORTANT: Forbidden Python packages

The final submission is not allowed to use these imports:

```
from scipy.integrate import solve_ivp
from scipy.linalg import solve_continuous_are
```

You must implement your own version of continuous LQR solver and IVP solver. Your LQR solver must be based on "direct method" discussed in class and the "numbook". Thus, you need to implement these two functions:

1. `solve_continuous_are`
2. `solve_ivp`

Make sure that the signatures of these functions and values returned correspond to their usage in the file `modal_lqr.py`.

The Gradescope Autograder will call your implementation through functions:

1. `simulate_closed_loop`
2. `simulate_open_loop`

You must put your implementation in the file `student.py` in folder `src/python`.

### 11.2 Functions not to be modified

The autograder will call the function `build_model`. You should not modify this function, with one exception: if you are implementing the optional extension described below.

## Optional extensions

### 11.3 Viscosity

If desired, one may add viscous damping to the PDE:

$$u_{tt} = c^2 \Delta u - \gamma u_t + b(t)\psi(x, y), \qquad \gamma > 0.$$

Then the modal equations become

$$\ddot{q}_k + \gamma \dot{q}_k + \omega_k^2 q_k = \beta_k b(t).$$

This is not necessary for the project, but it can make the numerical simulations more forgiving.

### 11.4 Gaussian patch

You would use:

$$\psi(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{1}{2\sigma^2}\|(x,y)-(x_0,y_0)\|^2}.$$

This would be handled by the extra parameter to the model: $\sigma$. Note that the integral of $\psi$ is not 1, but as $\sigma \to 0$ this is a proper approximation to the $\delta$-function patch.

### 11.5 How to handle the extension, so that the autograder notices it?

The autograder hook is the function `build_model`. Add the parameter $\gamma$ and/or $\sigma$ to your model if you implement this optional extension. Autograder will call your `build_model`. Of course, you must adapt other functions that use the model, so that they properly handle the new parameter.